

A controlled small-language-model study

Parameter-matched SwiGLU under a fixed training budget

Individual coursework | WikiText-2 | Fixed BPE-2048 | September 2026

Abstract

We replace the supplied small GPT's GELU feed-forward sublayer with a near-parameter-matched SwiGLU network. Across two paired random seeds, mean full-validation BPB decreases from 2.07516 to 2.01698, a 2.80% relative reduction, at an identical budget of 9,830,400 training targets per run. Parameters increase by only 0.44%. The checkpoint selected exclusively on validation achieves 2.04783200 full-test FP32 CPU BPB, compared with 2.10270222 for the supplied baseline trained at seed 17 (2.61% lower). Results support a useful small-model improvement in this setting, rather than a general claim across tasks or training scales.

Task and experimental question

The objective is next-token prediction from random initialization using only the supplied training split. The question is whether replacing the baseline GELU feed-forward sublayer with a near-parameter-matched SwiGLU sublayer improves full-validation bits per byte (BPB) at an equal number of processed training targets. The final submitted model uses the same training-target budget as the baseline, so additional training is not responsible for the measured architectural comparison.

Protocol and constraints

The fixed protocol is **7506-mp1-wt2-v2**: vocabulary 2,048, independent causal windows of 256 targets, including the final short window. The model has no external retrieval assets, no cross-window state, and no evaluation network access. The supplied data, tokenizer, common utilities and scorer are unchanged.

Constraint	Required limit
CPU scoring time	At most 5 times the supplied baseline on the same machine
Peak evaluation RAM	At most 4 GiB
Uncompressed inference assets	At most 64 MiB

Metric

BPB = summed negative natural-log probability / $\ln(2)$ / raw UTF-8 byte count. Lower is better. The denominator follows the course protocol and includes the entire raw split. Validation has 376,599 scored targets and 1,148,007 bytes; test has 428,405 targets and 1,292,013 bytes.

One isolated architectural change

Baseline and candidate

The provided GPT has four blocks, width 128, four attention heads, learned absolute positions, pre-layer normalization, causal scaled dot-product attention, and tied input/output embeddings. These existing components are retained and are not claimed as new contributions.

Baseline: $\text{FFN}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2$, with inner width 512.

Candidate: $\text{FFN}(x) = W_d [\text{SiLU}(W_g x + b_g) * (W_v x + b_v)] + b_d$, with inner width 344. The multiplication is elementwise. A fused gate/value projection is split into two tensors.

SwiGLU follows Shazeer [1]. Its input-dependent multiplicative gate may improve the use of a limited feature budget. This is a hypothesis to test on this dataset, not a guarantee. Three projections require a narrower inner dimension than the baseline two-projection FFN: $3D_h$ approximately equals $8D$ squared, so h is approximately $8D/3$, rounded to a multiple of eight. Biases and rounding prevent an exactly equal parameter count.

Controlled comparison and ablation

Each paired run uses 1,200 updates, batch size 32, context 256, and 9,830,400 processed targets. Seeds 17 and 29 are repeated across both models. The data-start generator is seeded separately, so paired runs see the same sampled training windows. Model initialization follows the supplied normal initialization; new FFN tensors necessarily differ between architectures. The gate-disabled implementation is tested to reproduce the supplied baseline exactly under an identical seed, making the baseline the mechanism-removal ablation.

Training and checkpoint selection

The supplied recipe uses AdamW, peak learning rate 0.001, weight decay 0.1, 100-step warmup, cosine decay to 10% of peak, and gradient clipping at 1.0. Paired runs use the same training precision. Validation is evaluated in FP32 every 300 steps. The trainer now saves both the final checkpoint and the best validation checkpoint; fixed-budget comparisons use final checkpoints. Selection never uses test BPB.

A longer 4,800-step experiment was launched after the paired runs but produced no metrics or checkpoint before a reported system interruption. It contributes no result to the comparison. Further GPU training was stopped, and the final model remains a 1,200-step checkpoint; the interrupted attempt is disclosed in the run ledger.

Correctness

The original contract tests cover causal prefix independence, normalized finite probabilities, sample independence, reset between windows, finite nonzero gradients, and exact target coverage. An additional test verifies that gate removal equals the baseline. Protected files are checked against release SHA-256 hashes.

Results before the test set

Run	Params	Targets (M)	Final val. BPB	Best val. BPB	Train (s)
baseline-s17	1,088,256	9.83	2.07289	2.07289	20.7
baseline-s29	1,088,256	9.83	2.07743	2.07743	20.1
gated-s17	1,093,056	9.83	2.01735	2.01735	21.3
gated-s29	1,093,056	9.83	2.01662	2.01662	21.6

All values above are measured. Best-checkpoint BPB is reported separately from the final-checkpoint equal-budget comparison. Training time is as recorded by the trainer; intermediate validation and checkpoint work can affect wall-clock cost.

Interpretation

At seed 17, BPB decreases by 0.05554; at seed 29, it decreases by 0.06082. The consistent direction across the two paired seeds supports the mechanism, although two seeds are insufficient for a strong statistical-significance claim.

The gate-disabled branch is exactly the baseline, so the paired results also constitute a mechanism ablation. Only the FFN changes; attention, token budget, sampled windows, optimizer and learning-rate recipe remain fixed. The small parameter-count mismatch and differing FFN initialization are residual confounders.

Every completed run's best observed validation checkpoint is the final update 1,200. The best of the four candidates is gated seed 29. It was chosen before any test scoring; seed selection is disclosed and is not the claimed algorithmic contribution.

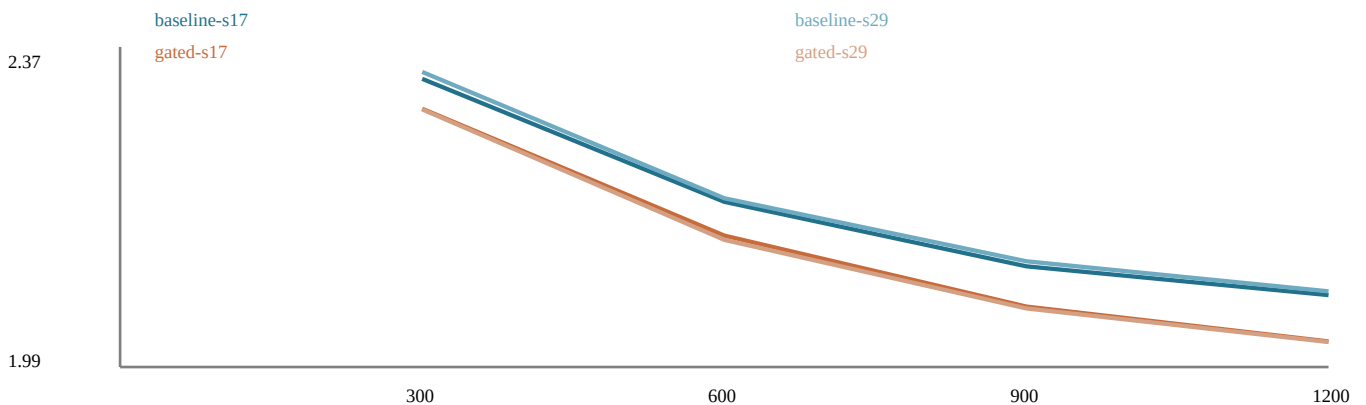


Figure 1. Full-validation BPB by update for the paired experiments; lower is better.

Reproduction and resource audit

The frozen submission is the gated model at seed 29, update 1,200. It was selected using the lowest full-validation BPB among the four completed paired runs. The unmodified baseline seed-17 checkpoint is the reference. Both were scored repeatedly only after the candidate was frozen; no test-dependent tuning followed.

Measured quantity	Baseline	Frozen candidate
Full-test FP32 CPU BPB	2.10270222	2.04783200
Median CPU scoring time (s)	11.024	11.311
Peak process RAM (GiB)	1.571	1.574
CPU time / baseline	1.000	1.026
Uncompressed inference files	-	5.302 MiB

Measurement procedure

The unchanged evaluator runs in fresh Python processes, with CPU, FP32 and four PyTorch threads. Three trials alternate baseline/candidate order. The comparison uses median scorer time, excluding loading. Both run at Below Normal process priority on Windows to reduce desktop disruption. Peak RAM covers loading and the evaluator process tree, including the Windows virtual-environment launcher and its Python child. At 20 ms intervals, psutil sums their peak-working-set counters (or current RSS if larger); the maximum sum is a conservative memory estimate. Absolute times are machine-specific. The reference is the original architecture trained at seed 17 for 1,200 steps, not a hard-coded instructor runtime.

Hardware: AMD Ryzen 7 5800H (8 cores / 16 logical processors), approximately 16 GiB physical RAM, NVIDIA GeForce RTX 3060 Laptop GPU with 6 GiB VRAM. GPU training uses PyTorch 2.7.1+cu126; CPU evaluation uses the CPU build of PyTorch 2.7.1; exact installed package versions are recorded in results/environment-cpu.txt and results/environment-gpu.txt.

Identity and assets

The final checkpoint is **submission/model.pt**. Its configuration and implementation module are embedded in the checkpoint. Source and checkpoint hashes were recorded in results/freeze.json before scoring the test split. The asset manifest counts the checkpoint, required model source, fixed tokenizer and configuration conservatively (audit metadata itself is excluded); no learned database or hidden external file is required.

Checkpoint SHA-256: 01a37e8f19ae0c312913f719bbe43eb8bfcd97fe3f3ca9014d39fe3e565e0e9

Cost disclosure

The four completed full runs processed 39,321,600 targets in 83.69 measured training seconds and 155.16 trainer-process seconds (including preparation/validation). A separate 10-step CPU installation smoke test processed 81,920 targets in 5.98 training seconds (60.70 process seconds). The interrupted extension has unmeasured overhead and no saved results. Dependency installation/download and report generation are excluded; no pretrained checkpoint or earlier model ancestry was used.

Limits, reproducibility and attribution

What the evidence establishes

The evidence is limited to WikiText-2, this tokenizer, a roughly one-million-parameter model, two seeds and a 1,200-update schedule. The curves still improve near the end, so this is not a claim of optimal training duration or best possible leaderboard score. Longer-budget results are unavailable.

Test improvement compares the validation-selected seed-29 candidate with baseline seed 17. It is not itself a paired same-seed architecture estimate; the two paired validation comparisons supply that evidence. Validation-based checkpoint/seed selection can still introduce selection bias.

CPU timing depends on background applications, process priority and hardware. Three trials summarize local runtime variation; they do not guarantee the exact instructor runtime. Peak memory uses conservative process-tree working sets, not GPU VRAM. Repeated test scores check determinism rather than estimating sampling uncertainty.

Reproduce the frozen score

Install Python 3.12, PyTorch 2.7.1 and the provided requirements. From code/, run:

```
python evaluate.py --checkpoint submission/model.pt
--device cpu --precision fp32 --threads 4 --split test
```

The command is a single shell command; the line break above is for layout. Use the returned **bpb** field. No retraining is necessary. REPRODUCE.md documents training, tests and resource measurement. The source archive includes the fixed benchmark, raw experiment metrics and reproduction commands.

AI assistance and reused work

The baseline, scorer, dataset packaging and tokenizer are supplied by the course. SwiGLU is an existing mechanism [1]. OpenAI Codex assisted with experimental design, implementation, tests, execution, analysis and drafting this report and the accompanying documentation. The student is responsible for personally reviewing and understanding the implementation before submitting; this report does not assert that this review has already occurred.

References

- [1] Noam Shazeer. GLU Variants Improve Transformer. 2020. <https://arxiv.org/abs/2002.05202>
- [2] DASE7506 MP1 course starter package. GUIDE.md, README.md, model.py and fixed evaluation protocol 7506-mp1-wt2-v2. 2026.
- [3] Stephen Merity et al. Pointer Sentinel Mixture Models. 2016. <https://arxiv.org/abs/1609.07843>. WikiText text is by Wikipedia contributors; retain the course README attribution to CC BY-SA 3.0 and GFDL.